02/Abril/2020

ESTRUTURA DE DADOS

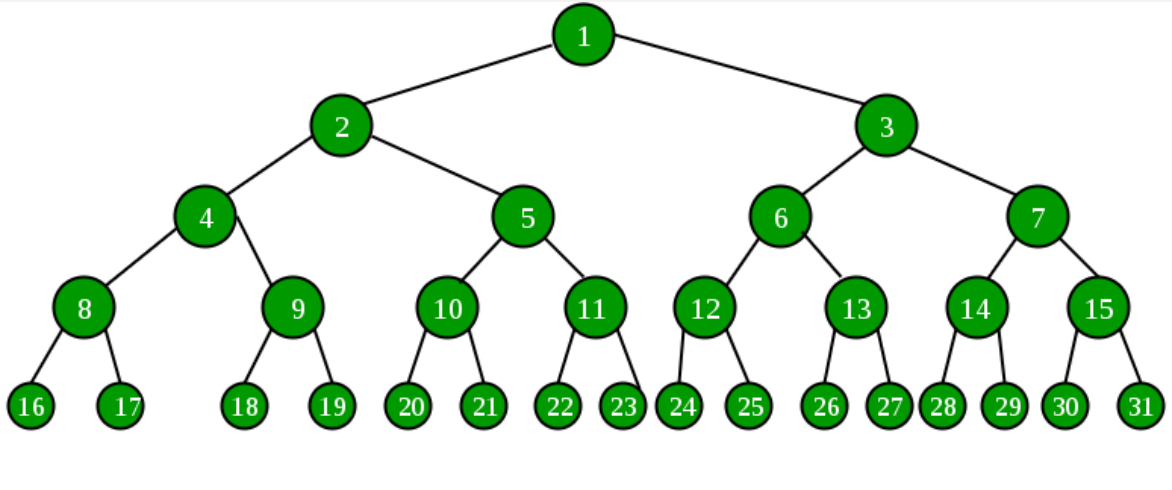
ÁRVORE BINÁRIA

Prof. Carlos Veríssimo

ÁRVORES BINÁRIAS

INTRODUÇÃO

- Uma árvore binária é uma estrutura de dados mais geral que uma lista encadeada
- Abordaremos algumas operações básicas sobre árvores binárias.



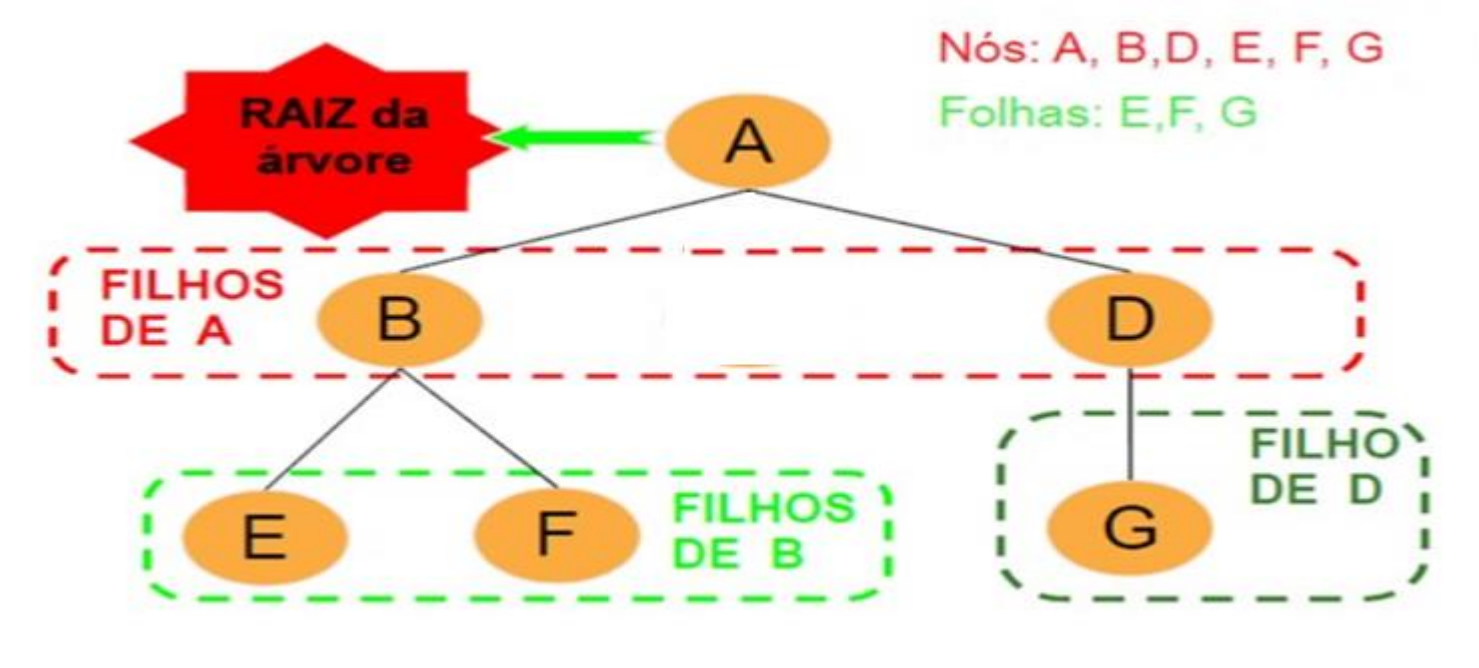
ÁRVORES BINÁRIAS

ÁRVORES

- Em computação , uma árvore é um modelo *abstrato* de uma estrutura hierárquica, não linear, construída composta por *nós* com relação de parentesco (pais e filhos). A forma mais natural definirmos uma estrutura de árvore é utilizando a recursividade.
- Uma árvore é composta por um conjunto finito de nós
- Sua composição : nó *r* (raiz), que contém zeros ou mais sub-árvores. Estas são filha de *r*.
- Nós com filhos são chamados de ***nós internos***
- Nós que não contém filhos, são chamados de ***nós externos*** (***folhas***)

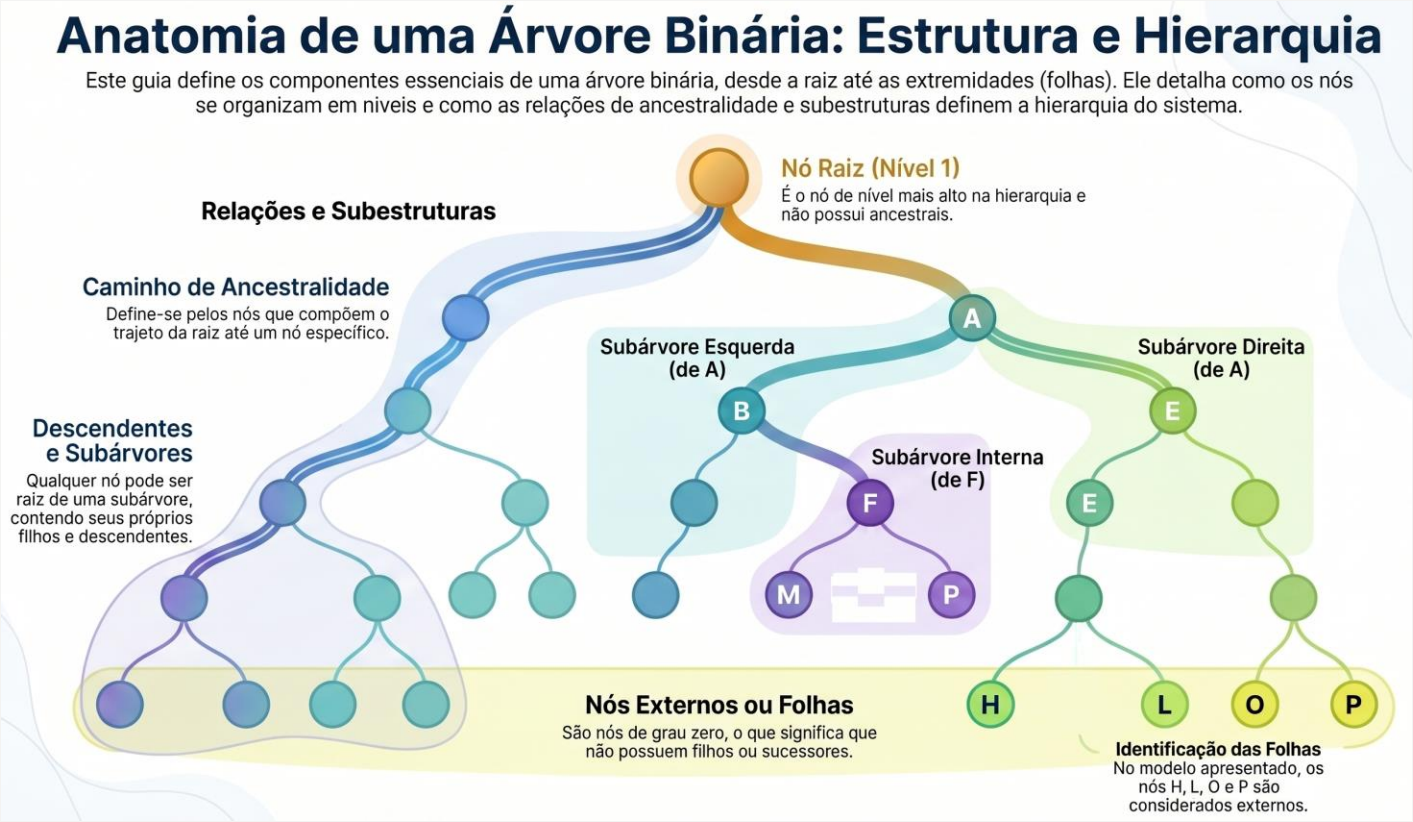
ÁRVORES BINÁRIAS

ÁRVORES



ÁRVORES BINÁRIAS

ÁRVORES BINÁRIAS



ÁRVORES BINÁRIAS

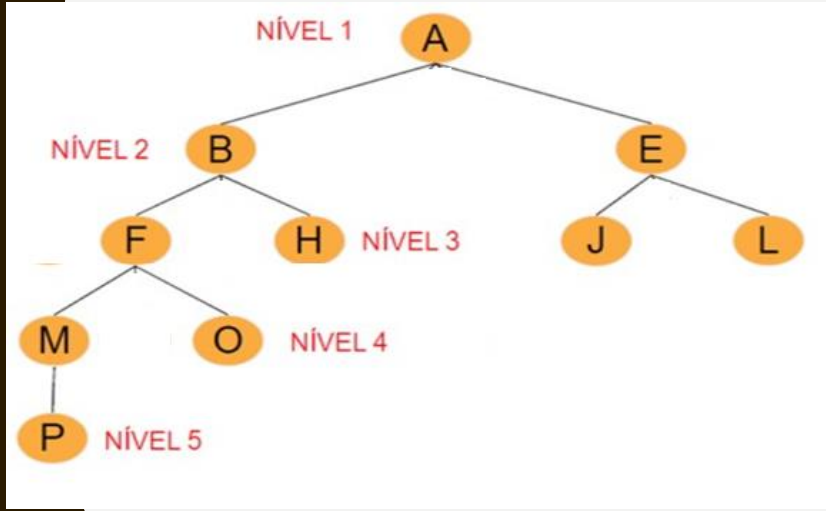
TERMINOLOGIA

Com base na estrutura da árvore fornecida, aqui está o detalhamento técnico da profundidade e altura de cada nó.

- **Profundidade:** É a distância do nó até a **raiz** (contando as arestas). A raiz sempre tem profundidade 0.
- **Altura:** É a distância do nó até a **folha mais profunda** abaixo dele. Um nó folha sempre tem altura 0.

ÁRVORES BINÁRIAS

TERMINOLOGIA



Nó	Nível	Profundidade	Altura
A	1	0	4
B	2	1	3
E	2	1	1
F	3	2	2
H	3	2	0
J	3	2	0
L	3	2	0
M	4	3	1
O	4	3	0
P	5	4	0

ÁRVORES BINÁRIAS

ÁRVORES BINÁRIAS -NÓS E FILHOS

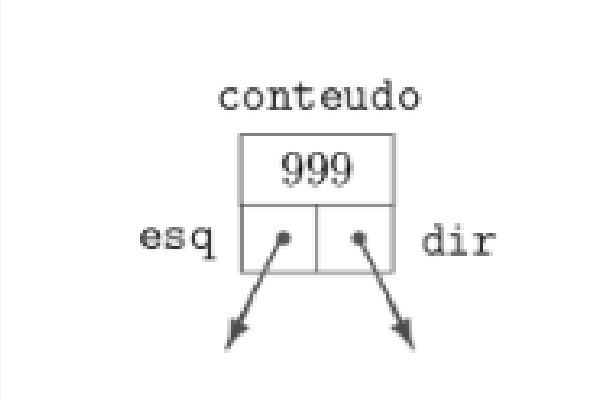
- Uma árvore binária (= *binary tree*) é um conjunto de registros que satisfaz certas condições. As condições não serão dadas explicitamente, mas elas ficarão implicitamente claras no contexto. Os registros serão chamados *nós* (poderiam também ser chamados *células*). Cada nó tem um endereço. Suporemos por enquanto que cada nó tem apenas três campos: um número inteiro e dois ponteiros para nós. Os nós podem, então, ser definidos assim:

```
typedef struct reg {
   int  conteudo; // conteúdo
   noh *esq;
   noh *dir;
} noh; // nó
```

ÁRVORES BINÁRIAS

NÓS E FILHOS

- O campo conteúdo é a "*carga útil*" do nó; os dois outros campos servem apenas para dar estrutura à árvore. O campo *esq* de cada nó contém NULL ou o endereço de outro nó. Uma hipótese análoga vale para o campo *dir*.



ÁRVORES BINÁRIAS

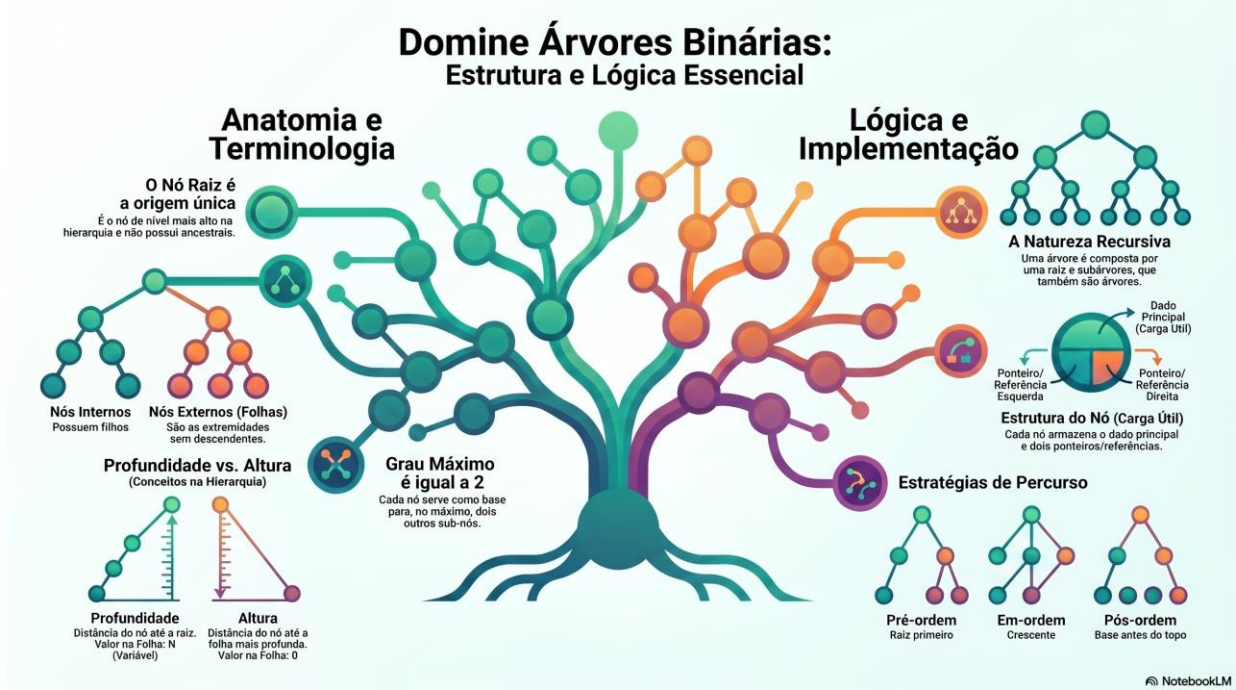
NÓS E FILHOS

- Se o campo esq de um nó P é o endereço de um nó E, diremos que E é o filho esquerdo de P. Analogamente, se P.dir é igual a D, diremos que D é o filho direito de P. Se um nó F é filho (esquerdo ou direito) de P, diremos que P é o pai de F. Uma folha *(= leaf)* é um nó que não tem filho algum.

ÁRVORES BINÁRIAS

NÓS E FILHOS

IMPLEMENTAÇÃO EM PYTHON



ÁRVORES BINÁRIAS

NÓS E FILHOS

IMPLEMENTAÇÃO EM PYTHON

Para implementar uma árvore em Python, transpomos a estrutura visual para a memória de forma muito mais simples que no C. Em vez de ponteiros manuais e endereços de memória, utilizamos a **Orientação a Objetos**.

Cada nó é um objeto independente que guarda referências para seus "filhos".

ÁRVORES BINÁRIAS

NÓS E FILHOS

IMPLEMENTAÇÃO EM PYTHON

A Estrutura do Nó (Classe)

Em Python, não usamos struct. Definimos uma classe onde o método inicializador configura o dado e as referências (que começam vazias como ***None)***:

```
class No:
    def __init__(self, dado):
        self.dado = dado
        self.esquerda = None  # Referência para o filho da esquerda
        self.direita = None   # Referência para o filho da direita
```

ÁRVORES BINÁRIAS

NÓS E FILHOS

IMPLEMENTAÇÃO EM PYTHON

A recursividade

A árvore é, por definição, uma estrutura recursiva: uma árvore é composta por uma raiz e subárvores, que por sua vez também são árvores. Por isso, funções recursivas são a forma mais natural e elegante de manipular esses dados.

Simplicidade: Resolver um problema na árvore inteira resume-se a resolvê-lo para o nó atual e "chamar a função novamente" para os filhos.

Navegação: Para chegar a um nó profundo, a função entra em si mesma sucessivamente até encontrar uma referência `None`.

ÁRVORES BINÁRIAS

NÓS E FILHOS

IMPLEMENTAÇÃO EM PYTHON

Percursos (Visitando os Nós)

Para listar os nós, usamos diferentes estratégias recursivas dependendo da ordem desejada:

Pré-ordem (Raiz, Esq, Dir): Visita a Raiz, depois mergulha na subárvore esquerda.

Em-ordem (Esq, Raiz, Dir): Essencial para listar os dados em ordem crescente/alfabética.

Pós-ordem (Esq, Dir, Raiz): Ideal para processar a base antes do topo (como calcular o tamanho total de pastas).

Implementar árvores sem recursividade exigiria estruturas auxiliares complexas. A recursividade aproveita a pilha de execução do sistema para rastrear o caminho, tornando o código limpo, legível e fiel à hierarquia visual do diagrama. Além disso, o Python gerencia a memória automaticamente, eliminando a preocupação com a exclusão manual de nós.